FSW Practical Exam

12 Simple Programs + Step-by-Step Execution Guide

HTML • CSS • Bootstrap • JavaScript • jQuery • AJAX • React • Node.js • Express • MongoDB

Quick exam strategy

Practise the browser-based programs first (2, 4, 7, 8), then the Bootstrap/jQuery programs. React, Express, AJAX and MongoDB require extra setup. Use the exact environment provided in your lab.

Important: Bootstrap and jQuery examples use online CDN links and need internet access. AJAX should be served from a local web server. React requires Node.js and Vite. MongoDB CRUD requires a running MongoDB server.

Program 1 — Responsive Website using CSS and Bootstrap

Aim: Create a responsive webpage using Bootstrap grid and CSS.

Filename: index.html

```
<!DOCTYPE html>
<!DOCTYPE html>
<html>
<head>
    <title>My Website</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <style>
        body { background: lightblue; }
        .box { padding: 30px; text-align: center; }
    </style>
</head>
<body>
    <nav class="navbar navbar-dark bg-dark">
        <div class="container">
            <a class="navbar-brand" href="#">My Website</a>
        </div>
    </nav>
    <div class="container mt-4">
        <div class="row">
            <div class="col-md-6 box bg-light">
                <h2>Welcome</h2>
                <p>This is my responsive website.</p>
                <button class="btn btn-primary">Click Me</button>
            </div>
            <div class="col-md-6 box bg-secondary text-white">
                <h2>About Us</h2>
                <p>Learning web development.</p>
            </div>
        </div>
    </div>
</body>
</html>
```

How to execute

1. Open Notepad or VS Code and paste the code.

2. Save as index.html (in Notepad, choose Save as type: All Files).

3. Double-click index.html to open it in Chrome. Internet is needed to load Bootstrap.

Expected output: A dark navbar and two sections; the sections stack on narrow screens.

Remember: col-md-6 makes each column half-width on medium screens and above.

Program 2 — CSS Box Model, Margin and Padding

Aim: Show content, padding, border and margin using CSS.

Filename: box.html

```
<!DOCTYPE html>
<html>
<head>
    <title>CSS Box Model</title>
    <style>
        .box {
            width: 200px;
            border: 5px solid black;
            padding: 20px;
            margin: 30px;
            background: lightblue;
        }
    </style>
</head>
<body>
    <h2>CSS Box Model</h2>
    <div class="box">This is a box.</div>
</body>
```

```
</html>
```

How to execute

1. Save the code as box.html.

2. Double-click the file or open it in Chrome.

Expected output: A light-blue box with a black border and space around it.

Remember: Box model order: Content → Padding → Border → Margin.

Program 3 — Bootstrap Navbar, Buttons and Form

Aim: Create a responsive form with Bootstrap classes.

Filename: form.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Bootstrap Form</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
    <nav class="navbar bg-dark navbar-dark p-3">
        <span class="navbar-brand">Student Form</span>
    </nav>
    <div class="container mt-4">
        <form>
            <label>Name:</label>
            <input type="text" class="form-control mb-3" required>
            <label>Email:</label>
            <input type="email" class="form-control mb-3" required>
            <button class="btn btn-success">Submit</button>
            <button type="reset" class="btn btn-danger">Reset</button>
        </form>
    </div>
</body>
</html>
```

How to execute

1. Save as form.html.

2. Open the file in Chrome. Internet is needed to load Bootstrap.

3. Try Submit with empty fields, then try Reset.

Expected output: A navbar, two input fields, and Submit and Reset buttons.

Program 4 — JavaScript User Input and Dynamic Output

Aim: Read a name from an input and display a greeting.

Filename: input.html

```
<!DOCTYPE html>
<html>
<body>
    <h2>User Input</h2>
    <input id="name" placeholder="Enter your name">
    <button onclick="show()">Submit</button>
    <p id="result"></p>
    <script>
        function show() {
            let n = document.getElementById("name").value;
            document.getElementById("result").innerHTML = "Hello " + n;
        }
    </script>
</body>
</html>
```

How to execute

1. Save as input.html.

2. Open it in Chrome, type a name and click Submit.

Expected output: If you enter Chetan, the page displays: Hello Chetan.

Remember: getElementById selects an element; .value reads input text.

Program 5 — jQuery hide() and show()

Aim: Hide and display an HTML heading using jQuery.

Filename: jquery.html

```
<!DOCTYPE html>
<html>
<head>
    <title>jQuery Hide Show</title>
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
</head>
<body>
    <h2 id="text">Welcome to FSW</h2>
    <button id="hide">Hide</button>
    <button id="show">Show</button>
    <script>
        $("#hide").click(function() {
            $("#text").hide();
        });
        $("#show").click(function() {
            $("#text").show();
        });
    </script>
</body>
</html>
```

How to execute

1. Save as jquery.html.

2. Open it in Chrome with internet access.

3. Click Hide, then click Show.

Expected output: The heading disappears and reappears when the buttons are clicked.

Remember: .click() handles a click event; .hide() hides; .show() displays.

Program 6 — AJAX: Fetch JSON Data

Aim: Fetch a JSON file and display its values on a webpage.

Filename: ajax.html and data.json

```
<!-- Save this as ajax.html -->
<!DOCTYPE html>
<html>
<body>
    <h2>Student Details</h2>
    <button onclick="loadData()">Load Data</button>
    <p id="result"></p>
    <script>
        function loadData() {
            let x = new XMLHttpRequest();
            x.onload = function() {
                let data = JSON.parse(this.responseText);
                document.getElementById("result").innerHTML =
                    data.name + " - " + data.course;
            };
            x.open("GET", "data.json");
            x.send();
        }
    </script>
```

```
</body>
</html>

/* Create a separate file named data.json with this content:
{
    "name": "Chetan",
    "course": "MCA"
}
*/
```

How to execute

1. Save the HTML part as ajax.html and the JSON object in a separate data.json file in the same folder.

2. Open that folder in VS Code and select Terminal → New Terminal.

3. Run: python -m http.server 8000

4. Open http://localhost:8000/ajax.html in Chrome and click Load Data.

5. If the Python command is unavailable, use another local web server available in your lab.

Expected output: Chetan - MCA.

Remember: AJAX uses XMLHttpRequest; JSON.parse converts JSON text into a JavaScript object. Do not open this example directly as file://.

Program 7 — JavaScript Form Validation

Aim: Check that a required name field is not empty.

Filename: validation.html

```
<!DOCTYPE html>
<html>
<body>
    <h2>Registration Form</h2>
    <form onsubmit="return validate()">
        <input id="name" placeholder="Enter name">
        <button type="submit">Submit</button>
    </form>
    <p id="result"></p>
    <script>
        function validate() {
            let n = document.getElementById("name").value;
            if (n.trim() == "") {
                document.getElementById("result").innerHTML =
                    "Name is required";
                return false;
            }
            document.getElementById("result").innerHTML =
                "Form submitted successfully";
            return false;
        }
    </script>
</body>
</html>
```

How to execute

1. Save as validation.html.

2. Open it in Chrome and click Submit with the field empty.

3. Enter a name and submit again.

Expected output: Shows “Name is required” when empty and “Form submitted successfully” when filled.

Remember: trim() also catches spaces. return false prevents actual form submission in this demo.

Program 8 — JavaScript Arrays and Objects

Aim: Store values in an array and an object, then display them.

Filename: array.html

```
<!DOCTYPE html>
<html>
<body>
    <h2>Student Details</h2>
    <p id="result"></p>
    <script>
        let subjects = ["Java", "Python", "HTML"];
        let student = {
            name: "Chetan",
            course: "MCA"
        };
        document.getElementById("result").innerHTML =
            "Name: " + student.name + "<br>" +
            "Course: " + student.course + "<br>" +
            "Subject: " + subjects[0];
    </script>
</body>
</html>
```

How to execute

1. Save as array.html.

2. Open the file in Chrome.

Expected output: Name: Chetan; Course: MCA; Subject: Java.

Remember: Arrays use numeric indexes starting at 0; objects use named properties.

Program 9 — React Component-Based User Interface

Aim: Create a simple React component.

Filename: src/App.jsx

```
function App() {
    return (
        <div>
            <h1>Welcome to React</h1>
            <p>This is my first component.</p>
        </div>
    );
}
export default App;
```

How to execute

1. Install Node.js if needed and open VS Code Terminal.

2. Run: npm create vite@latest myapp -- --template react

3. Run: cd myapp

4. Run: npm install

5. Replace the code in src/App.jsx with the code above.

6. Run: npm run dev and open the local URL shown in the terminal (often http://localhost:5173).

Expected output: A page showing “Welcome to React” and “This is my first component.”

Remember: A component is a reusable UI part. Keep the existing Vite main.jsx file.

Program 10 — React JSX and Props

Aim: Pass user details into a React component using props.

Filename: src/App.jsx

```
function User(props) {
    return (
        <div>
            <h2>User Details</h2>
            <p>Name: {props.name}</p>
            <p>Course: {props.course}</p>
        </div>
    );
}
function App() {
    return <User name="Chetan" course="MCA" />;
}
export default App;
```

How to execute

1. Use the React project created for Program 9.

2. Replace src/App.jsx with this code.

3. Run: npm run dev and open the URL shown in the terminal.

Expected output: User Details; Name: Chetan; Course: MCA.

Remember: Props pass data to a component. JSX allows HTML-like syntax in JavaScript.

Program 11 — Node.js and Express HTTP Server

Aim: Respond to an HTTP GET request using Express.

Filename: app.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
    res.send("Hello from Express Server");
});

app.listen(3000, () => {
    console.log("Server running on port 3000");
});
```

How to execute

1. Install Node.js and verify with: node -v and npm -v
2. Create and open a folder named expressapp in VS Code.
3. Run: npm init -y
4. Run: npm install express
5. Save the code as app.js.
6. Run: node app.js
7. Open http://localhost:3000 in Chrome. Stop the server with Ctrl+C.

Expected output: Browser displays “Hello from Express Server”; terminal says the server is running.

Remember: app.get handles GET requests; res.send sends the response.

Program 12 — Node.js and MongoDB CRUD

Aim: Connect to MongoDB and create, read, update and delete a document.

Filename: app.js

```
const { MongoClient } = require("mongodb");
const url = "mongodb://127.0.0.1:27017";
const client = new MongoClient(url);

async function main() {
    await client.connect();
    const db = client.db("college");
    const students = db.collection("students");

    // Create
    await students.insertOne({ name: "Chetan", age: 23 });

    // Read
    console.log(await students.find().toArray());

    // Update
    await students.updateOne(
        { name: "Chetan" },
        { $set: { age: 24 } }
    );

    // Delete
    await students.deleteOne({ name: "Chetan" });

    console.log("CRUD completed");
    await client.close();
}
main().catch(console.error);
```

How to execute

1. Install and start MongoDB locally; it must listen at 127.0.0.1:27017.
2. Create and open a folder named mongoapp in VS Code.
3. Run: npm init -y
4. Run: npm install mongodb

5. Save the code as app.js.

6. Run: node app.js

Expected output: Terminal prints the inserted student document and then “CRUD completed”. The document is updated and deleted.

Remember: CRUD = Create, Read, Update, Delete. This example needs a working MongoDB server.

Execution Cheat Sheet

HTML/CSS/Bootstrap/JavaScript/jQuery
Save .html → double-click or open in Chrome. CDN examples need internet.

AJAX
Keep ajax.html and data.json together → run python -m http.server 8000 → visit http://localhost:8000/ajax.html.

React
npm create vite@latest myapp -- --template react → cd myapp → npm install → edit src/App.jsx → npm run dev.

Express
npm init -y → npm install express → save app.js → node app.js → visit http://localhost:3000.

MongoDB
Ensure MongoDB server is running → npm init -y → npm install mongodb → save app.js → node app.js.

Last-minute memory list

• CSS Box Model: Content → Padding → Border → Margin.

• JavaScript: getElementById(), .value, innerHTML, function, if.

• jQuery: selector → click() → hide()/show().

• AJAX: XMLHttpRequest → open() → send() → JSON.parse().

• React: function component → JSX → export default; props pass data.

• Express: require → app.get → res.send → app.listen.

• MongoDB: insertOne → find → updateOne → deleteOne.